Rigorous upper bounds for the connective constants of self-avoiding walks on the diamond and face-centred cubic lattices

chemistry-open-problems · 25 October 2026 · record (preliminary)

Solver: Claude Opus 5.5 (effort xhigh); checks re-run by GPT-6-Astra (effort xhigh); verification pending.

Abstract. The diamond lattice (coordination 4, the classical lattice model of chains of tetrahedral sp^3 carbon) and the face-centred cubic lattice (coordination 12, a lattice model of dense polymer melts) are the two standard three-dimensional lattices of polymer physics besides the simple cubic one. The best published rigorous upper bounds for their self-avoiding-walk connective constants are those of Alm (1993): $\mu_{\mathrm{dia}} < 2.90940$ and $\mu_{\mathrm{fcc}} < 10.27777$; the exact enumerations of 2017 give the slightly better $\mu_{\mathrm{fcc}} \le 10.24059$ through the inequality of Ahlberg and Janson. We prove $\mu_{\mathrm{dia}} \le 2.8958880$ and $\mu_{\mathrm{fcc}} \le 10.1630815$. The proofs are finite forbidden-set automata whose paths dominate self-avoiding walks, as in our earlier work on the simple cubic lattice (result 9 of this collection): a state is a walk of loop size at most K up to the point group of the lattice, and the automaton may forget remembered sites along any self-avoiding path through contacts, the choice being optimised against the Perron vector. For the diamond lattice the two sublattices are handled by re-normalising the frame with the inversion through the midpoint of the last bond. Each bound is certified by an explicit automaton and an integer vector satisfying a Collatz–Wielandt inequality, checked in exact integer arithmetic by a checker written separately from the generator, and for small memories by a third, brute-force checker. Relative to the numerical estimates $\mu_{\mathrm{dia}} \approx 2.879$ and $\mu_{\mathrm{fcc}} \approx 10.0371$, the rigorous gaps shrink by 44 % and 38 %.

The question

Let c_n be the number of n-step self-avoiding walks (SAWs) starting at a fixed vertex of a vertex-transitive lattice. By submultiplicativity $c_{n+m} \le c_n c_m$, the limit $\mu = \lim_n c_n^{1/n} = \inf_n c_n^{1/n}$ exists [1]; it is the connective constant (growth constant), the effective coordination number of a long flexible chain with excluded volume on the lattice. Except for the hexagonal lattice it is not known exactly; Alm p. 116 [2] writes:

> *"For most lattices the value of μ is not known."*

Rigorous bounds are the only exact information. Alm's matrix method §6.3.3–6.3.4 and Tables 2, 11, 12 [2] gives the best published upper bounds for the face-centred cubic (fcc) and diamond ("tetrahedral") lattices,

$$\begin{aligned} \mu_{\mathrm{fcc}} &< 10.27777 && (n = 10,\ m = 4,\ 313 \times 313 \text{ matrix}),\\ \mu_{\mathrm{dia}} &< 2.90940 && (n = 22,\ m = 9,\ 1003 \times 1003 \text{ matrix}), \end{aligned}$$

improving the earlier 10.361 (Ahlberg and Janson) and 2.9175 (Wall and White). We found no later improvement (literature, citation databases and AI-campaign repositories; see `references.md`). The exact counts available today, c_n for $n \le 24$ on the fcc lattice

[3] [4] and $n \le 32$ on the diamond lattice [5] [6], give through the elementary bound $\mu \le (c_N/c_1)^{1/(N-1)}$ and the inequality of Ahlberg and Janson (1.6.1) [7] the values in Table 1: for the fcc lattice they beat Alm's bound, for the diamond lattice they do not. We are not aware that these derived values have been published.

	diamond	fcc
Alm's matrix method [2]	2.90940	10.27777
$(c_N/c_1)^{1/(N-1)}$	2.919627	10.244120
Ahlberg–Janson	2.917190	10.240586
numerical estimate	≈ 2.879	10.03705785(14)

Table 1 / Rigorous upper bounds before this work, and numerical estimates. Ahlberg–Janson: the unique positive root of $c_1 x^{N-1} = [c_N - (c_1-2)c_{N-1}]x + (c_1-2)[(c_1-1)c_{N-1} - c_N]$, as stated in (1.6.1) [7] (Alm §6.3.3 [2] applies it to the fcc lattice; for the diamond lattice the value is given for comparison only, it is above Alm's bound). Rows 2 and 3 are computed here from the published counts ($N = 32$ for diamond, $N = 24$ for fcc), rounded upwards. The diamond estimate comes from a linear ratio extrapolation of c_n, $n \le 32$ (non-rigorous); the fcc estimate is that of Clisby [8].

Theorem 2. $\mu_{\mathrm{dia}} \le 2.8958880$ and $\mu_{\mathrm{fcc}} \le 10.1630815$. More precisely, the forbidden-set automata of Section 3 with loop memory K certify the bounds of Tables 8 and 9; the stated ones come from $K = 26$ (348 320 130 states) for the diamond lattice and $K = 11$ (13 546 252 states) for the fcc lattice. Already $K = 16$ (diamond, $\mu_{\mathrm{dia}} \le 2.9068259$) and $K = 7$ (fcc, $\mu_{\mathrm{fcc}} \le 10.2402871$) improve on every previous bound.

The certificates are exact; what is not claimed is any estimate of μ itself, or of the behaviour of the bounds as $K \to \infty$.

1 The two lattices and their automorphisms

fcc. Vertices $\Lambda_{\mathrm{fcc}} = \{x \in \mathbb{Z}^3 : x_1 + x_2 + x_3 \text{ even}\}$, edges $\{x, x+u\}$ for u in the set U of the twelve vectors $(\pm1, \pm1, 0)$ and their coordinate permutations. Labels $0, \dots, 11$ of U are fixed in the order $(-1,-1,0)$, $(-1,1,0)$, $(1,-1,0)$, $(1,1,0)$, $(-1,0,-1)$, $(-1,0,1)$, $(1,0,-1)$, $(1,0,1)$, $(0,-1,-1)$, $(0,-1,1)$, $(0,1,-1)$, $(0,1,1)$.

Diamond. Let $a_0 = (1,1,1)$, $a_1 = (1,-1,-1)$, $a_2 = (-1,1,-1)$, $a_3 = (-1,-1,1)$ (the vectors $(\pm1,\pm1,\pm1)$ with product $+1$), $A = \{a_0, \dots, a_3\}$, and $\Lambda = \{x \in (2\mathbb{Z})^3 : x_1 + x_2 + x_3 \equiv 0 \pmod 4\}$. The vertices are Λ (type A) and $\Lambda + a_0$ (type B); the edges are $\{x, x + a_t\}$, $x \in \Lambda$. A vertex of type A has the neighbours $x + a_t$, one of type B the neighbours $x - a_t$. (Since $a_t - a_s \in \Lambda$, every $x + a_t$ with $x \in \Lambda$ is of type B.) This is the diamond lattice scaled by 4.

Let $\mathcal{G}$ be the group of the 48 signed permutation matrices, $\mathcal{G}^+ = \{g \in \mathcal{G} : gA = A\}$ (the 24 matrices with an even number of sign changes; it acts on A as the full symmetric group) and $\mathcal{G}^- = \{g \in \mathcal{G} : gA = -A\} = -\mathcal{G}^+$.

Lemma 3. (a) For $u \in U$ and $g \in \mathcal{G}$, $\gamma(x) = u + gx$ is an automorphism of the fcc lattice with $\gamma(0) = u$. (b) For $e \in A$ and $g \in \mathcal{G}^-$, $\gamma(x) = e + gx$ is an automorphism of the diamond lattice with $\gamma(0) = e$; it exchanges the two types.

Proof. (a) g permutes coordinates and changes signs, so it preserves $\mathbb{Z}^3$, the parity of $x_1+x_2+x_3$ and the set U; translation by $u \in \Lambda_{\mathrm{fcc}}$ preserves Λ_{fcc} and differences. So γ and $\gamma^{-1}(y) = g^{-1}(y-u)$ map vertices to vertices and edges to edges. (b) Every $g \in \mathcal{G}$ maps Λ onto itself: it preserves $(2\mathbb{Z})^3$, and changing the sign of an even coordinate x_a changes the sum by $2x_a \equiv 0 \pmod 4$. Hence $\gamma(\Lambda) = e + \Lambda = \Lambda + a_0$ (as $e - a_0 \in \Lambda$) and $\gamma(\Lambda + a_0) = \Lambda + e + ga_0 = \Lambda$ (as $ga_0 = -a_s$ for some s and $e - a_s \in \Lambda$). An edge $\{x, x+a_t\}$, $x \in \Lambda$, goes to $\{\gamma x, \gamma x + ga_t\}$ with γx of type B and $ga_t \in -A$: an edge. Finally $\gamma^{-1}(y) = g^{-1}(y-e) = e' + g^{-1}y$ with $g^{-1} \in \mathcal{G}^-$ and $e' = -g^{-1}e \in A$ is a map of the same form, so it also maps vertices to vertices and edges to edges. □

Both lattices are therefore vertex-transitive (indeed arc-transitive), and c_n does not depend on the starting vertex. We work in a *normalised frame*: the current vertex is the origin o, of type A for the diamond lattice; its neighbours are U (fcc) or A (diamond).

2 Forbidden-set automata

Definition 4. Let X be one of the two lattices and $N(o)$ the set of neighbours of o. A *forbidden-set automaton* consists of a finite set S of states, a finite set $F(s)$ of vertices for each $s \in S$, a start state s_0 with $F(s_0) = \emptyset$, and, for each $s \in S$ and each $e \in N(o)\backslash F(s)$, a successor $\sigma(s,e) \in S$ and an automorphism $\gamma = \gamma_{s,e}$ of X with $\gamma(o) = e$ and

$$\gamma(F(\sigma(s,e))) \subseteq F(s) \cup \{o\}. \tag{5}$$

Its transfer matrix is $A_{st} = \#\{e \in N(o)\backslash F(s) : \sigma(s,e) = t\}$, and $a_n = \sum_t (A^n)_{s_0 t}$.

$F(s)$ is a set of vertices known to be visited, seen from the current vertex; (5) says that the successor only remembers vertices that were really visited, the old current vertex included, seen from the new one.

Lemma 6 (domination). $c_n \le a_n$ for every $n \ge 0$.

Proof. Let $\omega = (\omega_0 = o, \omega_1, \dots, \omega_n)$ be a SAW. We define states s_i and automorphisms Γ_i with $\Gamma_i(o) = \omega_i$ and $\Gamma_i(F(s_i)) \subseteq \{\omega_j : j < i\}$: s_0 is the start state and $\Gamma_0 = \mathrm{id}$. Given s_i, Γ_i, let $e_i = \Gamma_i^{-1}(\omega_{i+1})$; it is a neighbour of o, and $e_i \notin F(s_i)$ because $\omega_{i+1} \notin \{\omega_j : j < i\} \supseteq \Gamma_i(F(s_i))$. Put $s_{i+1} = \sigma(s_i, e_i)$ and $\Gamma_{i+1} = \Gamma_i \circ \gamma_{s_i,e_i}$. Then $\Gamma_{i+1}(o) = \Gamma_i(e_i) = \omega_{i+1}$ and, by (5), $\Gamma_{i+1}(F(s_{i+1})) \subseteq \Gamma_i(F(s_i) \cup \{o\}) \subseteq \{\omega_j : j \le i\}$. The label sequence $(e_0, \dots, e_{n-1})$ is accepted by the automaton, and it determines ω: Γ_i is a function of $e_0, \dots, e_{i-1}$ and $\omega_{i+1} = \Gamma_i(e_i)$. So $\omega \mapsto (e_i)$ is injective. □

Lemma 7 (one-sided Collatz–Wielandt bound). If $V : S \to (0,\infty)$ and $\bar\lambda > 0$ satisfy $(AV)_s \le \bar\lambda V_s$ for all $s \in S$, then $\mu \le \bar\lambda$.

Proof. $A \ge 0$ gives $A^n V \le \bar\lambda^n V$, so $a_n = (A^n \mathbf{1})_{s_0} \le \bar\lambda^n V_{s_0} / \min V$; with Lemma 6, $c_n^{1/n} \le (V_{s_0}/\min V)^{1/n} \bar\lambda \to \bar\lambda$. □

Nothing about how the automaton was found matters for soundness: only $F(s_0) = \emptyset$, the inclusions (5) with automorphisms fixing $o \mapsto e$, $V > 0$ and $AV \le \bar\lambda V$. No Perron–Frobenius theory, irreducibility or aperiodicity is used.

3 The automata used

States. A state is a SAW $w = (v_l, \dots, v_1, v_0 = o)$ ending at the current vertex, with $F(w) = \{v_1, \dots, v_l\}$, stored newest-first as labels $t_0, \dots, t_{l-1}$: the step $v_{i+1} \to v_i$ has displacement u_{t_i} (fcc) or $(-1)^{i+1} a_{t_i}$ (diamond; the sign alternates because the types alternate along the walk and v_0 is of type A). Positions: $p_0 = 0$, $p_{i+1} = p_i - \text{displacement}_i$. Two walks are identified when they differ by an element of the point group of the normalised frame, $\mathcal{G}$ (fcc) or $\mathcal{G}^+$ (diamond), which acts on the labels by a permutation; the stored representative is the lexicographically smallest image of $(t_0, t_1, \dots)$. A walk is *admissible* when $l + \delta(w) \le K$, where $\delta(w)$ is the length of a shortest lattice path from v_0 to v_l avoiding $v_1, \dots, v_{l-1}$ (and $\delta = l$ for $l \le 1$): the walk can still be closed into a polygon of at most K steps (the notion of Pönitz and Tittmann [9], here on other lattices). S_K is the set of admissible walks up to symmetry; for the diamond lattice $l + \delta$ is even, so only even K are used.

Children. From w and $e \in N(o) \setminus F(w)$, the visited set seen from the new vertex and re-normalised by $\tau(x) = x - e$ (fcc) or $\tau(x) = e - x$ (diamond) is $Q = \{0\} \cup \{\tau(p_i) : 0 \le i \le l\}$; τ^{-1} is an automorphism of the kind in Lemma 3 (with $g = \mathrm{id}$, respectively $g = -\mathrm{id} \in \mathcal{G}^-$). Every self-avoiding path $0 = P_0, P_1, \dots, P_m$ with all $P_i \in Q$, read as a walk ending at P_0, has its forbidden set inside $\tau(F(w) \cup \{o\})$; if its canonical form is a state t, then $t = hP$ for some h in the point group and $\gamma = \tau^{-1} \circ h^{-1}$ satisfies (5). The generator enumerates the paths by a depth-first search through Q (pruned by $m + d(0, P_m) \le K$, which is non-decreasing along a path), takes at each leaf the deepest prefix that is a state, and discards candidates whose vertex set is strictly contained in another candidate's. The path that follows w backwards in time gives the child of Pönitz and Tittmann (delete the oldest sites until the walk is admissible); the others jump along *contacts* (non-consecutive vertices of w at distance one) and keep vertices close to the current one.

Choice of the children. Any choice gives a valid automaton. With

$$T(v)_s = \sum_e \min_{t \in C(s,e)} v_t$$

($C(s,e)$ the candidates for the step e from s) we run min-sum value iteration $v \leftarrow T(v)/\max T(v)$ (all candidates in memory) or, for the largest K, policy iteration with the candidate lists cached on disk; both converge to the same eigenvalue to 10^{-10} where both were run. The vector is normalised to $\max v = 1$, $V_s = \max(1, \lceil 2^{50} v_s \rceil)$, each child is the candidate of smallest integer weight, and the certified bound is P/Q with $Q = 10^7$ and $P = \lceil Q \max_s (AV)_s / V_s \rceil$, the maximum being computed exactly.

4 Certificates and checks

Format `LATCERT1` (little endian): the 8 bytes `LATCERT1`; $u32$ lattice (0 diamond, 1 fcc), $u32$ K; $u64$ N, P, Q; N strictly increasing $u64$ state codes; N weights V ($u64$); $N \times |N(o)|$ $u32$ child indices ($2^{32} - 1$ for forbidden steps). A code has bit 63 set, the walk length in bits 57–62 and the label t_i in bits $[b\,i, b\,i + b)$, $b = 2$ (diamond) or 4 (fcc), all other bits zero.

Checker (`src/latsaw_check.cpp`, written separately from the generator and sharing no code with it: no canonical forms, no search for children). It builds, for each $e \in N(o)$, the set Γ_e of maps $x \mapsto e + gx$, $g \in \mathcal{G}$, that send every vertex of the box $[-6,6]^3$ to a vertex and every edge at such a vertex to an edge, and checks $|\Gamma_e| = 48$ (fcc), 24 (diamond); by Lemma 3 these are automorphisms. Then:

- (C0) codes strictly increasing and decodable, $1 \le V \le 2^{60}$, $P, Q \ge 1$, the empty walk listed;
- (C1) for every listed s and every $e \in N(o)\backslash F(s)$: the listed child t exists and some $\gamma \in \Gamma_e$ maps $F(t)$ into $F(s) \cup \{o\}$, tested point by point;
- (C2) $Q \sum_e V_{\sigma(s,e)} \le P\, V_s$ for every s, in exact unsigned 128-bit arithmetic (sums of at most 12 weights $\le 2^{60}$ fit in 64 bits).

(C0)–(C2) are exactly the hypotheses of Lemmas 6 and 7, so a PASS proves $\mu \le P/Q$. The checker also reports the exact maximal ratio as a reduced fraction and, optionally, the path counts a_n of the certified automaton, compared with the exact c_n ($a_n \ge c_n$ must hold by Lemma 6; equality for small n is a test of the geometry).

Further checks. (i) `scripts/verify_small_bruteforce.py`, a pure-Python checker that finds the automorphisms from the vertex test and the neighbour function only, tests the inclusions with sets and the ratio with fractions, reproduces the exact maximal ratios for diamond $K \le 16$ and fcc $K \le 8$ (both already below the previous bounds). (ii) `scripts/tamper_test.py`: claimed P lowered by one unit (after setting P to the exact ceiling), one weight lowered by 10 %, a zero weight, a child pointer redirected or removed, the start state removed, two codes swapped, a label out of range: every corrupted copy is rejected. (iii) $a_n \ge c_n$ for all $n \le 32$ (diamond) and $n \le 24$ (fcc), with equality up to the n_0 of Tables 8 and 9. (iv) The generator in Pönitz–Tittmann mode (one child: the longest admissible suffix) gives the values of the column "P–T child" of Tables 8 and 9; the optimised children are never worse, and better for every $K \ge 8$ (diamond) and $K \ge 3$ (fcc).

5 Results

The exact maximal Collatz–Wielandt ratios of the certificates of Theorem 2, as reduced fractions printed by the checker, are

$$\text{diamond, } K = 26: \ 5502674668048/1900168345657,$$
$$\text{fcc, } K = 11: \ 3533716493477759/347701287175229.$$

Checking times: diamond $K = 24$: 47 s (3 threads); diamond $K = 26$: 200 s (4 threads, 10.9 GB of memory); fcc $K = 11$: 22 s (4 threads).

Restricted candidates for fcc, $K = 11$. On the fcc lattice the number of self-avoiding paths through the remembered set grows quickly (on average 85, 178 and 364 search nodes per (state, step) for $K = 7, 8, 9$), and the full candidate search at $K = 11$ would take several hours and a 37 GB candidate cache. For $K = 11$ the search only follows paths that move to an older remembered vertex or to the next younger one along the remembered walk. At $K = 9$ this restriction costs 0.0015 (10.19332 instead of 10.19181) and divides the search by four. At $K = 11$ it leaves 13.0 candidates per (state, step)

K	states	P–T child	optimised	certified	n_0	time
6	6	2.9477116	2.9477116	2.9477116	7	< 1 s
8	13	2.9366222	2.9363985	2.9363985	9	< 1 s
10	95	2.9240951	2.9229751	2.9229752	11	< 1 s
12	483	2.9174734	2.9163625	2.9163626	13	2 s
14	3 284	2.9119382	2.9107536	2.9107537	14	3 s
16	20 777	2.9080175	2.9068259	2.9068259	16	3 s
18	142 606	2.9048280	2.9036474	2.9036474	17	4 s
20	970 961	2.9023067	2.9011404	2.9011404	19	13 s
22	6 831 896	2.9002137	2.8990683	2.8990683	19	2 min
24	48 381 572		2.8973474	2.8973475	20	14 min
26	348 320 130		2.8958879	**2.8958880**		3 h 24 min

Table 8 / Diamond lattice. States: $|S_K|$. P–T child: the automaton with the child of Pönitz and Tittmann only; optimised: candidate children, value iteration (policy iteration for $K = 26$); both floating-point eigenvalue brackets, for comparison only. Certified: the bound P/Q, $Q = 10^7$, accepted by the checker. n_0: $a_n = c_n$ for $n \le n_0$ (and $a_n \ge c_n$ for all $n \le 32$; not computed for $K = 26$, where the count would need about 22 GB of memory). Time: generation, wall clock on a shared 12-thread machine (6 threads; 5 for $K = 26$).

(1.57×10^9 in all, a 6.8 GB cache); the initial policy, taken from the $K = 10$ certificate, has growth rate 10.17194, and ten passes of policy iteration bring it to 10.1630814. Any set of candidates gives a valid automaton; the certificate is checked exactly like the others.

6 Remarks and limitations

- *Where the gain comes from.* For the diamond lattice the optimised children gain only about 0.0011 at equal K over the child of Pönitz and Tittmann (contacts are rare on a lattice of girth 6); the automaton with the Pönitz–Tittmann child alone already beats Alm's bound from $K = 16$ (2.9080175). The improvement over Alm is mostly memory: his largest matrix has order 1003, the number of 9-step walks up to symmetry, while the certificate for $K = 26$ has 3.5×10^8 states. For the fcc lattice the optimised children matter much more (0.074 at $K = 3$, 0.034 at $K = 10$), because the lattice is full of triangles and the shortcuts keep the nearby vertices.

- *Convergence.* Successive gains are 0.0066, 0.0056, 0.0039, 0.0032, 0.0025, 0.0021, 0.0017, 0.0015 (diamond, $K = 10 \to 12 \to \cdots \to 26$) and 0.083, 0.053, 0.038, 0.027, 0.021, 0.017 (fcc, $K = 4 \to 5 \to \cdots \to 10$). A geometric extrapolation would not reach the estimates 2.879 and 10.0371: the convergence is slower than geometric. The Pönitz–Tittmann automaton with memory K accepts only walks without loops of length $\le K$, so its growth rate lies below the connective constant μ_K of walks with memory K, which tends to μ Lemma 1.2.3 [7]; in every computed case our optimised automata do better than the Pönitz–Tittmann ones. No rate of convergence is known, and no extrapolated value is claimed.

- *Resources.* The state space grows by a factor of about 7 per step of K (diamond: per $K \to K+2$; fcc: per $K \to K+1$). Diamond $K = 28$ would need about 2.4×10^9 states; fcc $K = 12$ about 10^8 states with $\sim 10^{12}$ search nodes. Both are beyond the

K	states	P–T child	optimised	certified	n_0	time
3	3	10.6568542	10.5825757	10.5825757	3	< 1 s
4	11	10.4891374	10.4138209	10.4138209	4	< 1 s
5	59	10.3923176	10.3311091	10.3311092	5	2 s
6	409	10.3301080	10.2785015	10.2785015	6	2 s
7	3 046	10.2865691	10.2402871	10.2402871	7	3 s
8	23 927	10.2542897	10.2128081	10.2128082	8	10 s
9	193 631	10.2293849	10.1918123	10.1918123	8	1 min
10	1 604 728	10.2095815	10.1752875	10.1752876	9	15 min
11	13 546 252		10.1630814	**10.1630815**	10	42 min

Table 9 / Face-centred cubic lattice; columns as in Table 8 ($a_n \geq c_n$ checked for $n \leq 24$). For $K = 11$ the candidate children are restricted (see the text) and the choice is made by policy iteration.

shared machine used here (23 GB of memory).

- *The fcc counts.* The exact counts of 2017 alone already improve Alm's fcc bound (Table 1); our $K = 7$ certificate (3 046 states) is the first one below that value, $K = 8$ is checked by all three programs.
- *Equality $a_n = c_n$.* The certified automata count the walks exactly up to n_0 steps; at larger n they also count some non-self-avoiding walks. With optimised children n_0 can be smaller than $K - 1$ (diamond $K = 22$: 19), because the automaton may forget more than necessary when this lowers the weight.
- *Status.* The bounds are certified by exact certificates and a checker written separately from the generator; they have not yet been verified by a second agent. The certificates for diamond $K \geq 20$ and fcc $K \geq 10$ (up to 11.1 GB for diamond $K = 26$) are kept outside the repository with their SHA-256 sums and are regenerated deterministically by `scripts/reproduce_large.sh`.

References

[1] J. M. Hammersley. Percolation processes. II. The connective constant. *Mathematical Proceedings of the Cambridge Philosophical Society*, **1957**.
DOI: 10.1017/S0305004100032692 1

[2] Sven Erick Alm. Upper bounds for the connective constant of self-avoiding walks. *Combinatorics, Probability and Computing*, **1993**.
DOI: 10.1017/S0963548300000547 1, 2

[3] Raoul D. Schram, Gerard T. Barkema, Rob H. Bisseling and Nathan Clisby. Exact enumeration of self-avoiding walks on BCC and FCC lattices. *Journal of Statistical Mechanics: Theory and Experiment*, **2017**.
DOI: 10.1088/1742-5468/aa819f 2

[4] OEIS Foundation Inc. Sequence A001336: number of n-step self-avoiding walks on the f.c.c. lattice. The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A001336. b-file for $n \leq 24$ (from Schram et al. 2017), accessed 25 October 2026. **2026**. 2

[5] OEIS Foundation Inc. Sequence A001394: number of n-step self-avoiding walks on the diamond lattice. The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A001394. b-file for $n \leq 32$, accessed 25 October 2026. **2026**. 2

[6] Anthony J. Guttmann. On the critical behaviour of self-avoiding walks. II. *Journal of Physics A: Mathematical and General*, **1989**.
DOI: 10.1088/0305-4470/22/14/027 2

[7] Neal Madras and Gordon Slade. The Self-Avoiding Walk. Reprinted 2013. Birkhäuser, **1993**.
DOI: 10.1007/978-1-4614-6025-1 2, 6

[8] Nathan Clisby. The growth constant for self-avoiding walks on the fcc and bcc lattices. *Journal of Physics A: Mathematical and Theoretical*, **2022**.
DOI: 10.1088/1751-8121/aca189 2

[9] André Pönitz and Peter Tittmann. Improved upper bounds for self-avoiding walks in $\mathbb{Z}^d$. *Electronic Journal of Combinatorics*, **2000**.
DOI: 10.37236/1499 4